



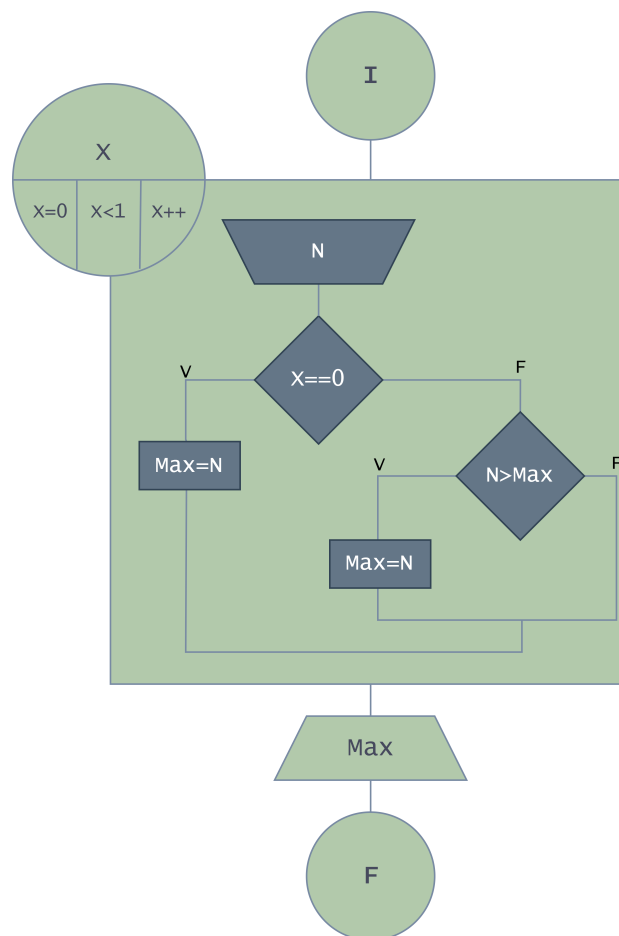
Guía de Ejercicios (Ciclo FOR) - Resueltos

Leer atentamente y resolver en diagrama de flujo.

Requerido: Videos E1 a E8 (especialmente este último) de la serie para [Aprender Programación Desde Cero](#).

¡No olvides que no alcanza con leer la problemática una sola vez! Dedícale todas las lecturas y el análisis que sea requerido. Repasá con los videos! Consultá las dudas. Y desde ya, tenés que usar el ciclo FOR en esta guía.

1. Hacer un programa que solicite el ingreso de 10 números y que muestre el mayor de ellos por pantalla. Solo se debe emitir UN valor por pantalla.



En este caso tenemos que hacer un programa para ingresar 10 números y vamos a buscar el mayor de ellos. Lo primero a realizar es plantear la estructura principal de nuestro programa, la que nos va a permitir ingresar esos 10 números. Para esto armamos un ciclo configurado para realizar 10 iteraciones y dentro del mismo ponemos una instrucción de ingreso. Esto nos da como resultado un programa que pide 10 valores. No hace nada con ellos, en cada vuelta solo pide uno nuevo y el valor anterior se pierde... ¿Cómo seguimos?

Vamos a guardar el primer ingreso como el mayor para obtener nuestro primer número de comparación con los restantes ingresados. Recordemos que X en el ciclo FOR es una variable que lleva el índice, es decir, la cuenta de las vueltas que va dando nuestro ciclo y la configuración que estamos usando es la estándar para hacer un Ciclo For, con lo cual: en la primera vuelta "x" va a valer cero, en la segunda vuelta "x" va a valer 1, en la tercera vuelta "x" va a valer 2 y así sucesivamente hasta que corte. Siguiendo este mismo ejemplo: ¿Te acordás en qué valor de "x" y en qué número de vuelta sería el corte? Analízalo para refrescar el funcionamiento del ciclo.

Bien. Nuestro programa inicia, luego entra en el ciclo y pedimos el ingreso de un número. Para guardar este primer número ingresado como el máximo, le preguntamos al ciclo si estamos en la primera vuelta. ¿Cómo Hacemos esto? Como dijimos, la variable "x" del FOR lleva el índice, entonces si "x" está en 0, estamos frente a la primera vuelta. En el programa lo preguntamos de la siguiente forma "**x == 0**", es decir, si "x" vale 0. Si este IF

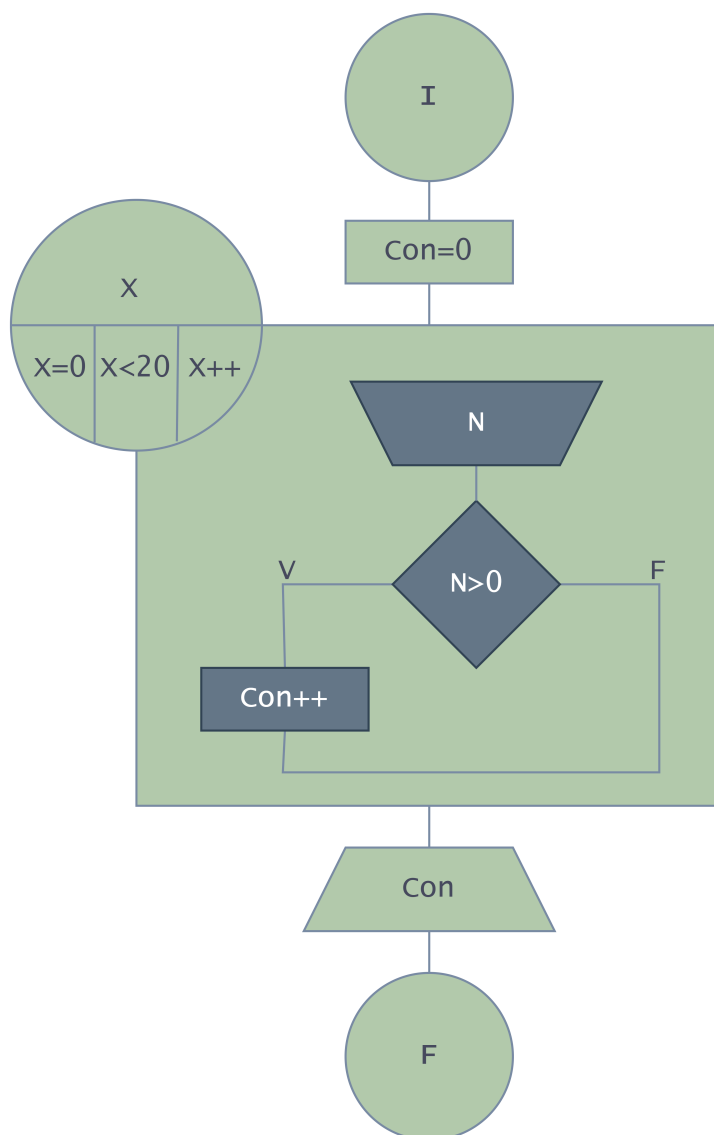
da verdadero quiere decir que estamos en la primera vuelta. Guardamos en la caja de proceso el número ingresado como el **MAX**.

¿Por qué guardamos este primero número como máximo si en realidad aún no sabemos cuál es el máximo?

Porque para poder comparar, necesitamos arrancar desde algún lado, entonces usamos la “estrategia” de: el primero que tengo, es el máximo, por la sencilla razón de que es el único que tengo... HASTA AHORA... luego veremos el resto...

Bueno... una vez que ya tenemos nuestro primer ingreso guardado, tenemos que empezar a buscar un máximo con los siguientes números que se ingresen. Para esto nuestro FOR volverá a ciclar (girar, iterar) y en la segunda vuelta, cuando se tope nuevamente con la condición “**x == 0**”, nos dará falso, porque “x” ahora vale 1, entonces de la segunda vuelta en adelante cada vez que se tope con este IF nos dará falso. Y es aquí donde empezamos a comparar estos números ingresados; con la utilización de otro IF preguntamos si el número ingresado es mayor al máximo que ya tenemos guardado de la primera vuelta de la siguiente manera “**N > MAX**”. Si la condición da verdadero, quiere decir que el valor dentro de “**N**” es mayor que el valor que habíamos guardado antes y que ahora se aloja en “**MAX**”, entonces en la siguiente instrucción, el contenido de “**N**” va a pisar el contenido de “**MAX**” dejando de esa manera un nuevo valor máximo. En el caso de que no sea mayor, no hará nada. Luego de iterar 10 veces, nuestro ciclo terminará y, fuera del mismo, mostraremos por pantalla el MÁXIMO obtenido.

2. Hacer un programa que solicite 20 números y calcule y emita por pantalla cuántos son positivos (mayores a cero). Se debe mostrar un solo valor: el conteo final.



Este programa es parecido al que realizamos en guías anteriores para determinar si un número era positivo o negativo, la diferencia es que lo hacíamos con un solo número y ahora lo realizaremos con 20.

La forma de pensar este ejercicio comienza parecido al ejercicio anterior: tengo que pedir 20 números, entonces armo un ciclo de 20 y armo la instrucción que me permite pedir UN valor, pero como va dentro del ciclo, se ejecutará 20 veces. Como resultado: un programa que pide 20 valores. Perfecto.

En este caso vamos a tener que *contar* una cantidad. Esto es algo que hemos hecho en el pasado y para poder hacerlo tenemos que usar una variable especial que, de especial, lo único que tiene es el nombre que le damos, ya que en el fondo no deja de ser una variable común y corriente: el contador.

¿Qué pasa con los contadores? Son variables a las que les damos un valor inicial (las “inicializamos”) que generalmente va a ser cero, pues contamos “1, 2, 3, 4...” y para eso tenemos que arrancar en cero, ¿cierto? Como en la vida...

Okey, como vamos a contar, creamos nuestra variable **“CON”** (cuyo nombre podría ser cualquier otro, no te olvides) y la inicializamos en cero. ¿Dentro o fuera del ciclo? Bueno, acá tengo que analizar: ¿cuántas veces se va a ejecutar esa instrucción si la pongo fuera del ciclo? ¿Y si la pongo dentro? ¿Y cuántas veces quiero que se ejecute? ¿Necesito que se vuelva a cero a cada rato o eso lo quiero hacer una sola vez?

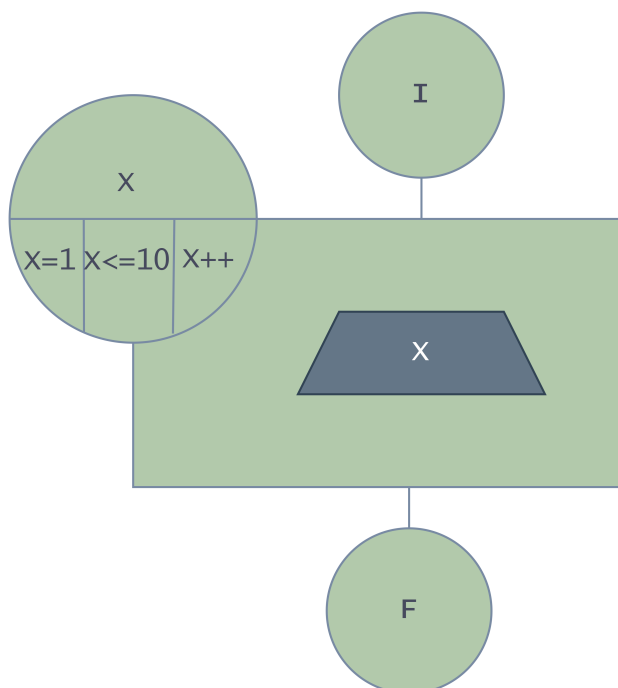
Perfecto, una vez con el contador inicializado, con el programa que ya pide los 20 números, ¿qué nos resta hacer? Analizar cada valor y ver si es positivo o no. Si es, cuento y sino, nada.

¿Cómo sé si un valor es positivo? Tiene que ser mayor a cero. NO cero, NO negativo. Mayor a cero. Entonces podemos preguntar si **“N > 0”**. Si es verdadero, es positivo y si es falso, entonces no lo es. ¿Y si es cero? No nos importa, la consigna en este caso no pide hacer nada con valores cero o negativos, así que los descartamos.

Volvemos al verdadero: si es verdadero, cuento. ¿Cómo puedo contar? A **“CON”** le asigno lo que tiene **“CON”** más 1, por ejemplo: **“CON = CON + 1”**. También puedo usar el operador de incremento, el “más más” y acortar la instrucción. Entonces nos queda **“CON++”**.

Una vez que terminamos de analizar todos los números, el ciclo concluye, mostramos en pantalla el conteo realizado.

3. Hacer un programa para mostrar los números del 1 al 10. No se debe realizar ningún pedido de datos.



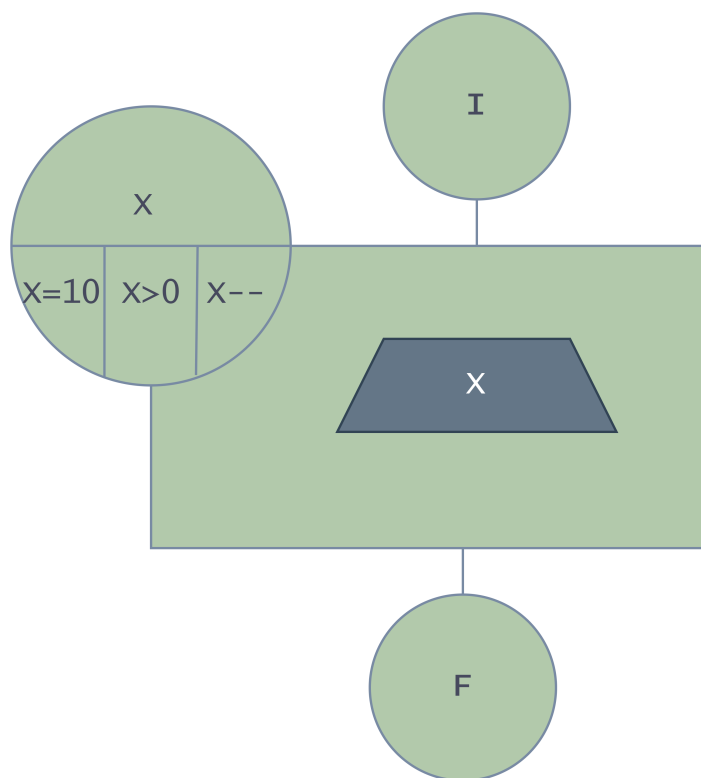
Para este programa vamos a jugar con la configuración del ciclo. No es la única manera de hacerlo, también podríamos hacer un contador que vaya incrementando su valor en cada vuelta (quizá fuiste por esa alternativa), pero si nos ponemos a pensar, ya tenemos una variable que hace esto y es nuestra variable “x”, que inicia con un valor y este se va incrementando en cada vuelta del ciclo.

En este ejercicio no tenemos que hacer ningún pedido de datos, es solo configurar el ciclo y algunas instrucciones. Entonces haremos lo siguiente: inicializamos “x” en 1 (distinto a lo clásico que es comenzar en

cero) y en la condición le decimos que mientras el valor de "x" sea menor o IGUAL a 10, siga dando vueltas; y, por último, le decimos que en cada vuelta "x" incremente su valor en 1.

Bien, dentro del ciclo imprimimos el valor de X en pantalla, en la primera vuelta "x" vale 1 por la configuración inicial que le dimos, en la segunda vuelta "x" se incrementó a 2 y así sucesivamente hasta llegar a 10, como resultado tendremos en pantalla 1,2,3...10. Cuando "x" vale 11 es cuando este ciclo va a concluir. ¿Lo viste? Analízalo.

-
4. Hacer un programa para mostrar los números del 10 al 1. No se debe realizar ningún pedido de datos.



Para este programa utilizamos la misma lógica que en el ejercicio anterior, pero al revés. Nos pide que mostremos los números de forma descendente del 10 al 1, sin pedir ningún dato.

Vamos directamente a la configuración del ciclo: en este caso le damos un valor inicial a nuestra variable "x" de 10.

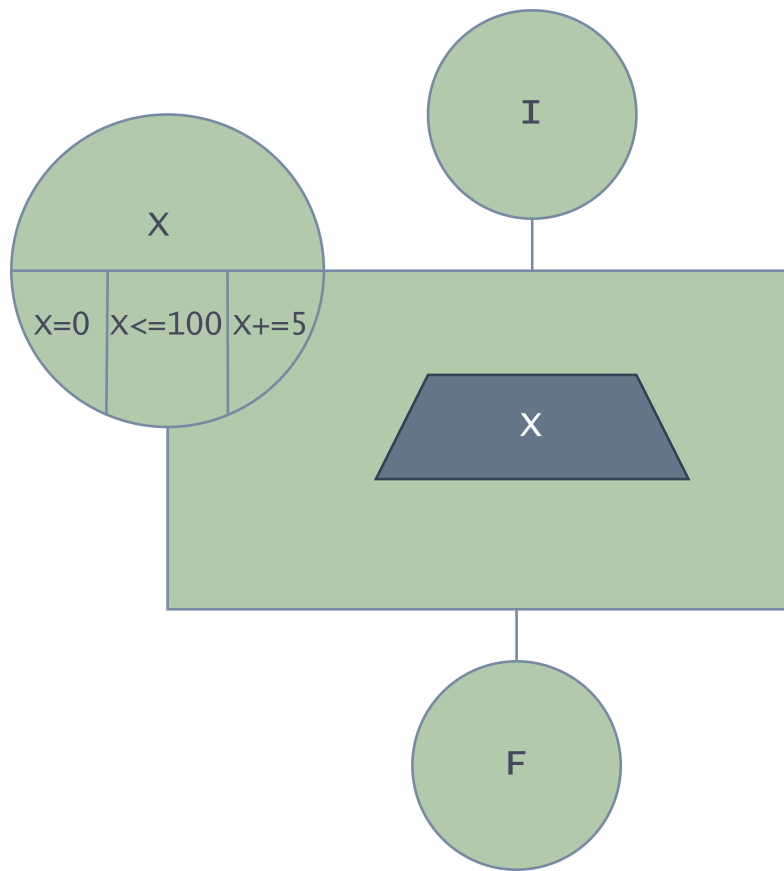
"**x = 10**". En nuestra condición le decimos que nuestro ciclo de vueltas mientras que "x" sea mayor a 0. "**x > 0**". Por último, le decimos que "x" vaya disminuyendo su valor en 1. "**x--**".

Analicemos esta configuración: entramos al ciclo y mostramos en pantalla el valor de "x", que en la primera vuelta es un "10".

Segunda vuelta: Ahora le dijimos que "x" disminuya en 1 su valor, entonces nos queda "x" en 9 y nuestra condición

"**x > 0**" pregunta ¿El valor de "x" es mayor a 0? En esta segunda vuelta, la respuesta es sí. Entonces da otra vuelta más y nos muestra en pantalla el 9. Esto se repite hasta llegar al 0. Cuando "x" valga 0, nuestra condición va a decir: "che, mirá, 0 no es mayor a 0", así que salimos del ciclo. Entonces finaliza el ciclo y termina nuestro programa. En pantalla tendremos los números del 10 al 1.

5. Hacer un programa que muestre los números del 0 al 100 de 5 en 5. Ejemplo: 0, 5, 10, 15, 20...100.



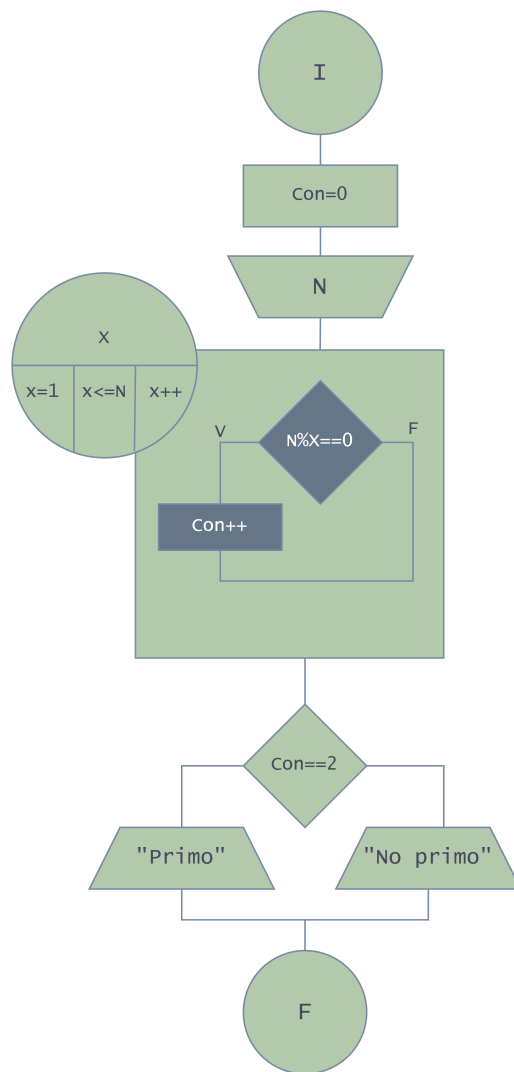
Bueno, para este ejercicio tenemos que mostrar los números del 0 al 100 pero de 5 en 5. Al igual que venimos trabajando en ejercicios anteriores, vamos a jugar con la configuración del ciclo.

Veamos la configuración del ciclo: pasamos a darle un valor inicial a “x = 0”, le decimos que mientras el valor de “x” sea menor o igual 100, siga ciclando; y por último, que incremente su valor en 5. Y no hay mucho más que esto. Nuestro programa inicia y vamos mostrando en pantalla el valor de “x”. En la primera vuelta obtenemos un 0, en la segunda vuelta un 5, porque recordemos que se incrementó el valor de “x” en 5: “**x+=5**” (también se puede escribir “**x = x+5**”); y así continuará ciclando hasta que llegue al 100.

Por la configuración que tenemos en nuestra condición, cuando el valor “x” supere el valor 100, sale del ciclo y termina nuestro programa.

6. Hacer un programa que solicite UN número y luego calcule y emita un cartel aclaratorio si el mismo es primo o no es primo.

Nota: un número es primo cuando es divisible únicamente por 1 y por sí mismo.



Bueno, para este ejercicio vamos a averiguar si el número ingresado es primo o no lo es, cómo hacemos esto? Primero tenemos que saber cuándo un número es primo, básicamente cuando dicho número es divisible por 1 y sí mismo, ósea obtenemos un resto 0 luego de hacer dicha división.

Bien, comenzamos creando un contador inicializado en 0, que este nos va a servir más adelante para poder determinar cuantas veces fue divisible, ósea si fue divisible 2 veces fue primo.

Luego pasamos a pedir el ingreso del número, lo hacemos fuera del ciclo porque solo pedimos un número para evaluar si es primo o no.

En la creación del ciclo inicializamos X en 1 porque lo vamos a utilizar para hacer nuestra primera división dentro del ciclo, en nuestra condición decimos que cicle mientras que el valor de X sea menor o igual a N, ósea que mientras que el valor de X sea menor o igual al número ingresado está todo bien. Por último, configuramos que X incremente su valor en 1 **x++**.

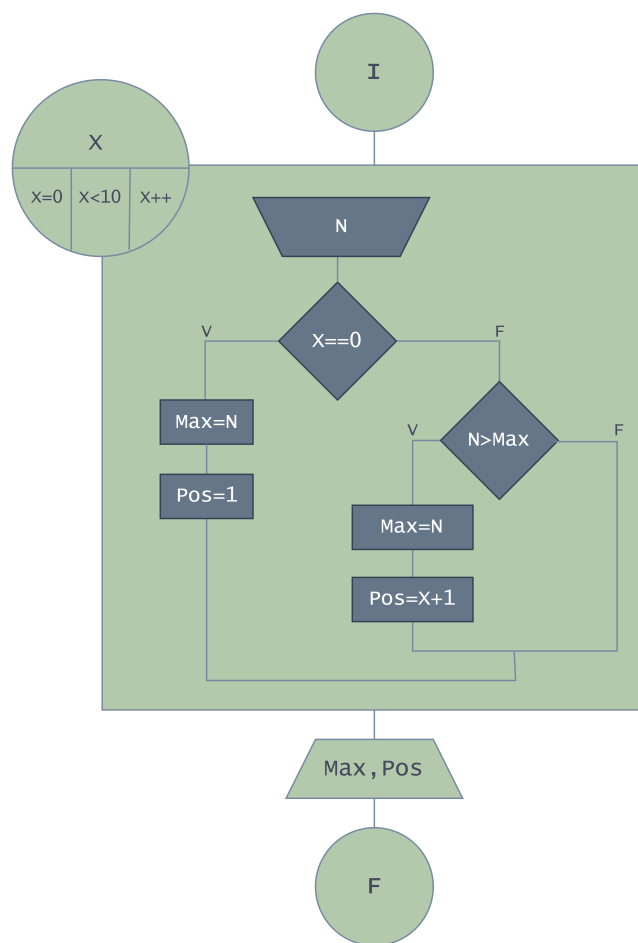
Bien, ahora dentro del ciclo creamos un if en el cual nuestra condición nos dice lo siguiente "Si la división entre el número ingresado y el valor de X actual da como resto 0 vamos al verdadero ". Si va al verdadero incrementamos nuestro contador, ósea **CON++**. En el caso de que fuera al falso no hacemos nada.

Tenemos que tener en cuenta que nuestro ciclo va a girar tantas veces como el número ingresado y el único valor que va a cambiar dentro de la división va a ser el valor de X, ya que se incrementa en cada vuelta. Pongamos un ejemplo de que se ingresó un 4, primer vuelta X vale 1. Entonces se divide 4 entre 1, en la segunda vuelta 4 entre 2 y así sucesivamente hasta que llegue a la vuelta 4 y divida 4 entre 4. Esto lo hacemos para buscar cuantas veces nos dio resto 0 al realizar estas divisiones, ósea desde 1 hasta el número que se haya ingresado.

Por último, al salir del ciclo, preguntamos si el valor de nuestro contador es igual a 2. **IF (CON == 2)**. Si es igual a 2 es porque fue primo, toma este camino del verdadero y mostramos en pantalla "PRIMO", "en el caso de que contador contenga un valor diferente a 2, va a tomar el camino del falso y mostramos en pantalla "NO PRIMO".

Una vez que se mostró el mensaje en pantalla, termina nuestro programa.

7. Hacer un programa que solicite 10 números y luego mostrar por pantalla el máximo de ellos y la posición en la que fue ingresado.



Este programa nos pide el ingreso de 10 números, mostrar el mayor número ingresado y en que posición se ingresó.

Bien, vamos a empezar utilizando una configuración estándar del Ciclo For y a diferencia de como veníamos trabajando que pedíamos el número afuera del ciclo, esta vez lo vamos a pedir dentro del ciclo porque necesitamos que cicle 10 veces para poder ingresar los 10 números, se va a ingresar un número por cada vuelta de For y para llevar la posición en que se ingresan vamos a utilizar nuestra variable X que como sabemos lleva el índice de las vueltas.

Empieza nuestro programa, entra al ciclo FOR, nos pide que ingresemos el primer Número, después preguntamos si estamos en la primera vuelta del ciclo utilizando el condicional $X == 0$, si X vale 0 significa que estamos en la primera vuelta del ciclo y para qué queremos saber si estamos en la primeravuelta? Bueno, esto nos interesa saberlo para poder guardar el primer número ingresado como el Máximo, nos va a servir para poder tener un Número “base” para poder compararlo con los siguientes ingresos.

Bien en la primera vuelta cuando se tope con el IF ($X == 0$) nos va a dar verdadero porque en la primera vuelta X vale 0, una vez dentro de este camino, guardamos el número ingresado como el máximo **MAX = N**, y le damos la posición de 1 porque estamos en la primera vuelta.

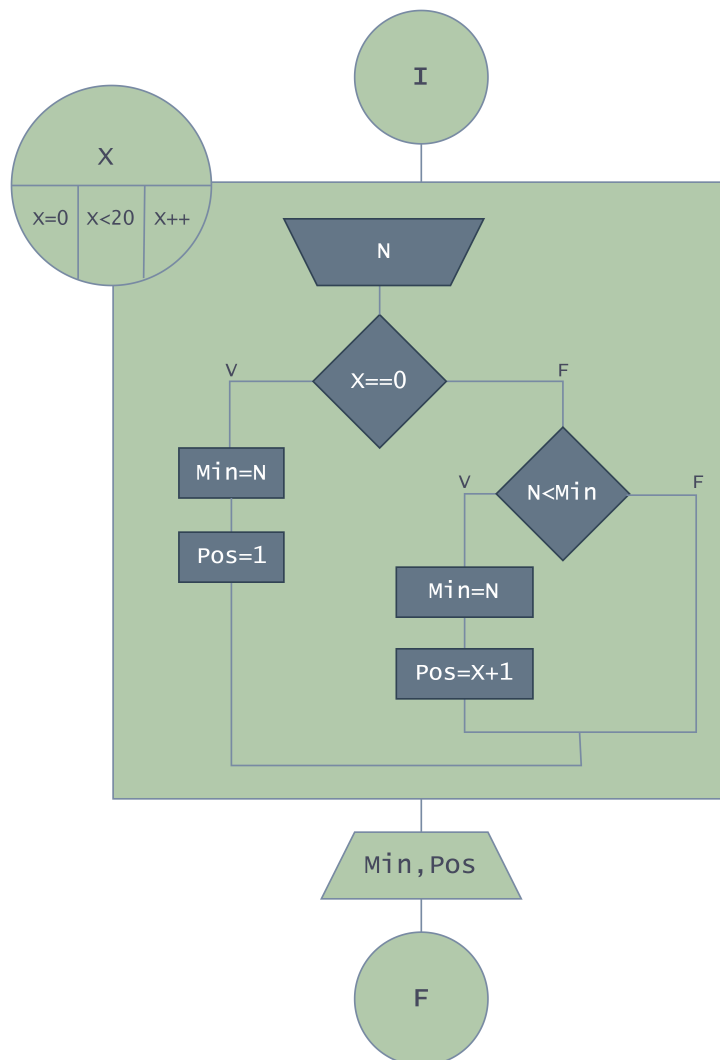
En la segunda vuelta y las que restan para completar las 10 vueltas cada vez que se tope con el IF ($X == 0$) nos va a dar falso porque X después de la primera vuelta ya se incrementa y deja de valer 0.

Bueno, ahora solo queda comparar los números que se vayan ingresando con nuestro máximo **MAX**. Para esto creamos otro IF ($N > \text{Max}$) Donde estamos preguntando si el número que se ingresó en esta vuelta es mayor al Máximo que ya tengo guardado. Si es mayor, va a tomar el camino del verdadero y reemplazamos el valor de **MAX** por el número ingresado. **MAX = N**, para saber en qué posición estamos utilizamos la variable X, como bien sabemos este va llevando el índice de las vueltas, pero tenemos un pequeño problema y es que tienen un desfase de valores nuestra posición con el índice de X, ya que X arranca en 0, entonces en nuestra vuelta 1 X vale 0, ¿cómo acomodamos esto? Simple, incrementado X de la siguiente forma **X + 1**, ahora nuestra variable de posición nos quedaría **POS = X + 1**.

En el caso de que el número ingresado no sea mayor al valor que tenemos guardado en **MAX** no hacemos nada, ya que solo nos pide guardar el mayor y la posición. Al salir del ciclo, mostramos en pantalla el máximo y la

posición de este, con esto terminaría nuestro programa. Si no entendiste bien lo del desfasaje de las posiciones, repasá el video donde Maxi lo explica a detalle. Clic [aquí](#) para verlo

8. Hacer un programa que solicite 20 números y luego mostrar por pantalla el menor de ellos y la posición en la que fue encontrado.

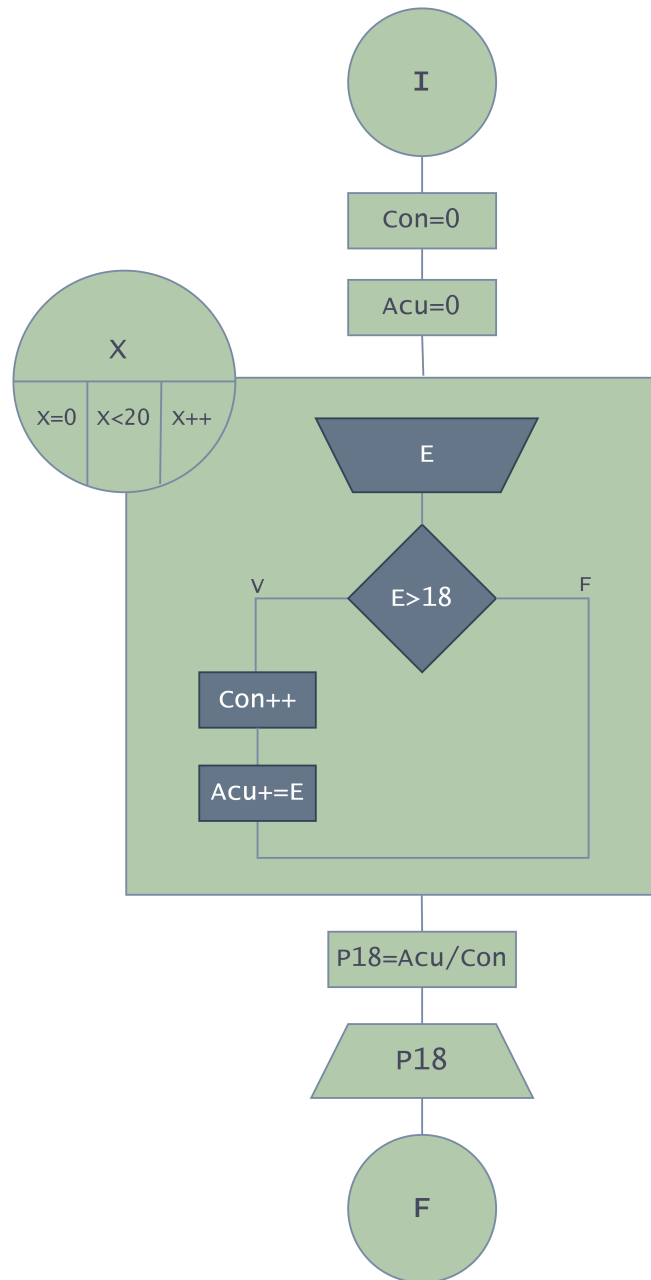


Para este ejercicio tenemos que hacer lo mismo que en el ejercicio anterior, pero en vez de buscar el Máximo número ingresado vamos a buscar el mínimo, esto lo hacemos realizando 2 cambios.

Uno renombrar nuestra variable anterior MAX por MIN a referencia que ahora buscamos un mínimo. Recordá que es importante darles un nombre o etiqueta apropiada a nuestras variables.

El paso dos sería modificar la condición del IF ($N > MAX$) por IF ($N < MIN$), en este caso vamos a preguntar si el número ingresado es menor al número que tenemos guardado en nuestra variable MIN nos quedaría **IF ($N < MIN$)** y listo estos serían los 2 cambios a realizar, después aplica la misma explicación que hicimos en el ejercicio anterior.

9. Hacer un programa que solicite 20 edades y luego calcule el promedio de edad de aquellas personas mayores a 18 años.



Para este programa vamos a primer tener en cuenta que nos pide el ingreso de 20 edades, para esto ya tenemos que tener en mente la configuración que vamos a usar para crear nuestro ciclo FOR, por otro lado, vamos a tener que discriminar por edades ingresadas en este caso que sean mayores a 18 y por último saber como se saca un promedio esto ya lo hicimos en la Guía 1 ejercicio 6 anda a repasarlo si no lo recordás.

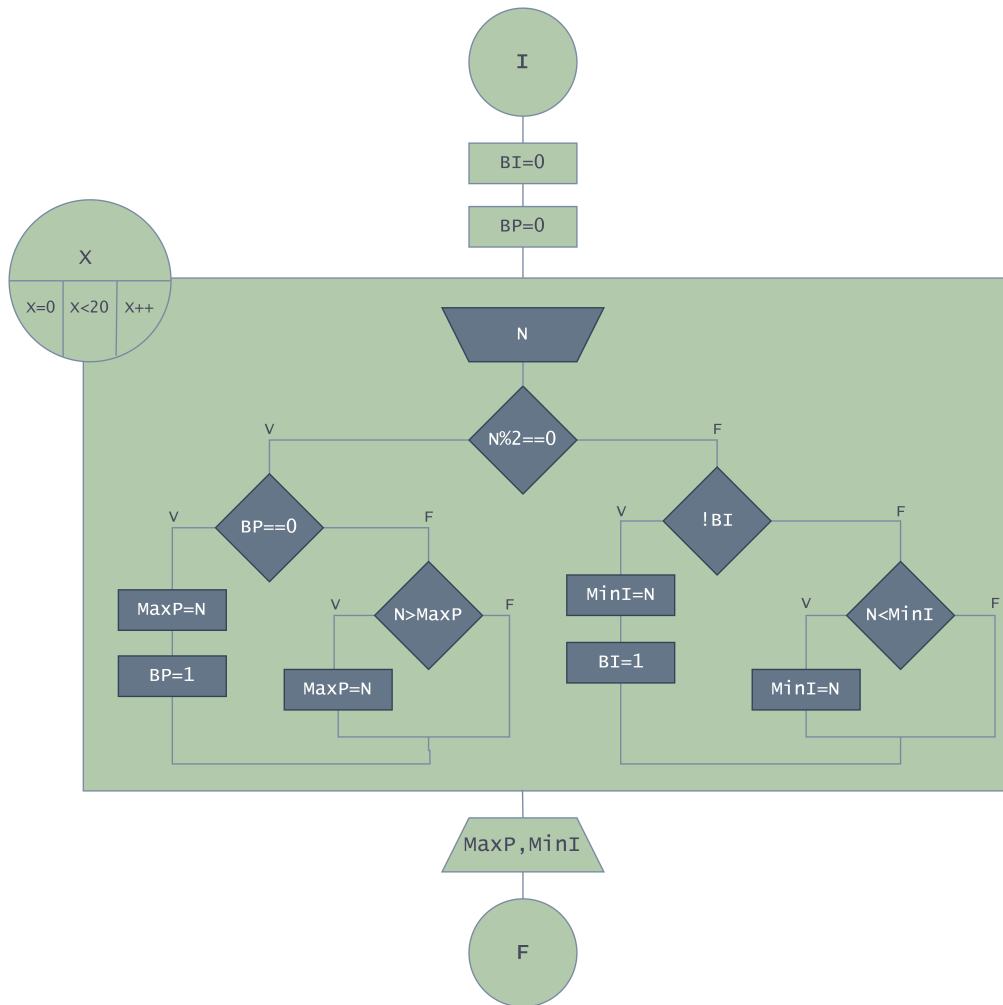
Bien, comienza nuestro programa y creamos 2 variables, una va a ser un contador la cual nos va a servir para sumar las edades mayores a 18, por otro lado, un acumulador donde vamos a acumular todas las edades que fueron mayores a 18.

Entramos al ciclo y pedimos el ingreso de edad que se va a guardar en **E**, seguido a esto preguntamos si la edad ingresada es mayor a 18 utilizando el IF ($E > 18$) en el caso de que sea mayor tomará el camino del verdadero y ahora incrementamos **CON++** y acumulamos la edad ingresada en **CON**, fijate que lo hacemos de la siguiente forma **"+= "** con esto le decimos acumúlame lo que tiene edad en **ACU**.

Una vez que termine de iterar nuestro Ciclo sale del programa y procedemos hacer el cálculo para sacar el promedio, en este caso el resultado obtenido lo guardamos en la variable **P18**. Nos queda de la siguiente Forma $P18 = ACU / CON$.

Por último, mostramos en pantalla el promedio **P18**.

10. Hacer un programa que solicite 20 números y luego emitir por pantalla el máximo de los números pares y el mínimo de los números impares.



Bueno para este ejercicio tenemos que saber cuándo un número par o impar, esto ya lo hicimos en la Guía 2 ejercicio 5 no te acordás anda a verlo. Dato nuevo que vamos a aprender es el tipo de dato Booleano "BANDERA". El Bool es un tipo de dato que admite 1 o 0 equivalentes a verdadero o falso.

Para este programa empezamos creando 2 variables del tipo Bool una BI "Bool de impares" y BP "Bool de Pares", las inicializamos en 0 esto significa que tiene un False dentro, estas banderas las vamos a utilizar para poder guardar nuestro primer número PAR ingresado y el primer número IMPAR ingresado.

La configuración del ciclo estándar, en este caso, va a ciclar 20 veces.

Bien, una vez dentro del Ciclo For, pedimos que se ingrese un número y se va a guardar en **N**. Luego tenemos que discriminar los números en PARES e IMPARES, nos va a quedar bien diferenciados ambos como podés ver en el diagrama. Para saber si es Par o no lo consultamos en el IF ($N \% 2 == 0$) Si el resto da 0 va a tomar el camino del verdadero, si no el camino del falso.

En ambos casos sea verdadero o al falso nos encontramos con un IF el cual vamos a utilizar para guardar el primer MAXIMO PAR o el primer MINIMO impar, analicemos ambos casos.

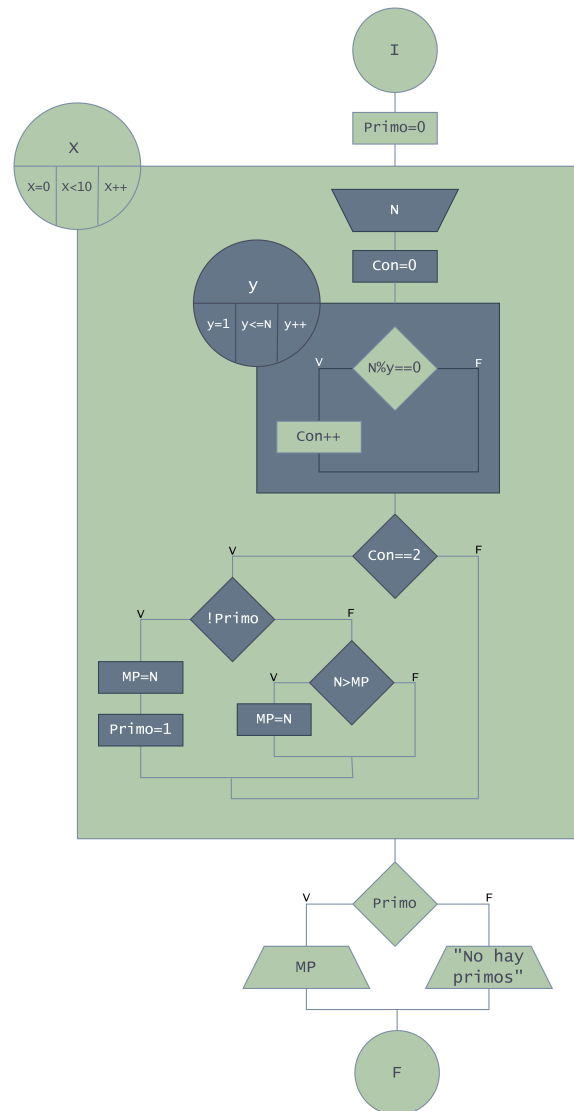
Primer caso: Supongamos que se ingresó un número PAR y va hacia el verdadero, nos topamos con el IF (**BP == 0**) Qué quiere decir esta condición? Esto nos está diciendo, si BP "Bool de Pares" contiene un 0, vamos al verdadero, entonces toma esta camino y guardamos el primer par ingresado en MAXP (**MAXP = N**) y ahora le cambiamos el valor de BP a 1. Para qué? Para que la próxima vez que se ingrese un número Par y se vuelva a topar con este IF tome el camino del falso y pregunte si este PAR que entro ahora es mayor al que ya teníamos guardado IF (**N > MAXP**). Si es mayor lo guardamos como el nuevo **MAXP** si no lo es, no hacemos nada.

Segundo Caso: Ahora supongamos que se ingresó un número impar, bueno va a tomar el camino del Falso y nos topamos con el siguiente IF (**!BI**) Y esto que es? Se te escapó un signo de exclamación? No, si miraste bien la tablita de operadores de la Guía anterior vas a darte cuenta de que estamos NEGANDO la condición del IF, esto quiere decir SI NO BANDERA, entonces le damos vuelta la condición como BI tiene un 0, ósea un FALSO, al darlo vuelta nos queda en un verdadero! Una vez que estamos acá, guardamos el número ingresado como

el primer mínimo IMPAR y le cambiamos el valor a **BI** por un 1. En la próxima que se vuelva a ingresar un número impar y se tope con nuestra condición negada va a tomar el camino del Falso, por qué? Porque le cambiamos el valor a 1 ósea verdadero y al negarla, la invertimos y se va para el falso. Si estás así como what's this !! Te dejo el minuto exacto donde maxi lo explica [link](#). Después igual que en el primer caso, cada vez que entre un IMPAR se va a ir para el falso comparando los números impares ingresados con nuestro MinImpar.

Después que salga del FOR, solo nos queda mostrar en pantalla el MAXP y el MinI y habrá terminado nuestro programa.

11. Hacer un programa para ingresar 10 números y luego calcule y emita el mayor de los primos de la lista. En caso de no haber ningún número primo, deberá aclararlo con un cartel.



Para este programa vamos a utilizar ciclos anidados, es decir, un ciclo dentro del otro. Un ciclo principal que nos va a servir para poder hacer el ingreso de 10 números y el ciclo que tenemos anidado nos va a servir para determinar si el número ingresado es primo.

Empezamos creando una bandera inicializada en 0 que como ya hemos visto nos va a servir para guardar el primer número primo que se ingrese.

Una vez dentro del Ciclo principal FOR X, pedimos el número a evaluar y creamos una variable de contador inicializada en 0, como ya sabemos si este contador nos da como resultado 2 es porque este número fue primo. La inicializamos dentro del Ciclo para que en cada vuelta que de el Ciclo principal este contador se vuelva a reiniciar a 0.

Bien, en el Ciclo For y. Lo que hacemos es ir dividiendo el número ingresado por el valor que va teniendo x en cada vuelta hasta llegar a dividirse por sí mismo, esto es lo mismo que hicimos con anterioridad para buscar un número primo.

Al salir de este Ciclo evaluamos si el contador es igual a 2, en el caso que sí, sabemos que el número ingresado fue primo, toma el camino del verdadero y nos topamos con el IF(! PRIMO) esta condición está negando a primo como vimos en el ejercicio anterior al estar inicializado en 0 ósea falso y negarlo nos da verdadero! Con esta condición lo que hacemos es poder guardar el primer número par ingresado. Guardamos el número ingresado en MP y cambiamos nuestra bandera a 1 para que en las próximas vueltas tome el camino del falso en todas las vueltas.

Bueno, a partir de la segunda vuelta del Ciclo se va a repetir todo el proceso anterior con la diferencia que ahora en el IF (! PRIMO) va a tomar el camino del falso y evaluar si el número par ingresado es mayor al que guardamos primero, en el caso de que lo sea pisa este valor $MP = N$, en caso de que no lo sea. No hacemos nada!

Por último, cuando ya termine de Ciclar 10 veces nuestro ciclo principal solo nos queda mostrar el Maximo Par si lo hubo o indicar que no se ingresó ningún número par, para esto vamos a reutilizar nuestra bandera PAR como sabemos esta bandera al iniciar el programa la inicializamos en 0, pero dentro del Ciclo si se ingresó un número par cambiamos su valor a 1. Entonces con esto sabemos que si su valor es 1 al final de programa es porque se ingresó un número par, si su valor es 0 es porque no hubo números pares.

¡Felicidades por completar la Guía 4 con éxito!

Es esta Guía aprendimos a utilizar el ciclo For "Ciclo Exacto "con su configuración estándar y también jugar con la misma.

Aprendimos un nuevo tipo de dato "Booleano" cómo crearlo y que esta puede alojar un 0 o 1 siendo 0 = false y 1 = true que es el mismo valor que maneja un if, si no tenés muy claro esto de tipos de datos repásate este [video](#).

Por otro lado, también aprendimos como invertir una condición utilizando el operador de exclamación para hacer una negación.

En los últimos ejercicios unimos todo lo aprendido en las Guías anteriores y lo nuevo de esta Guía, por eso recalcamos siempre tener en claro todos los conceptos anteriores antes de avanzar. En este punto los ejercicios se pueden resolver de diferentes formas, las propuestas aquí son una de ellas. Así que animáte a probar y resolverlos a tu forma.

Nos vemos en la próxima Guía donde vamos a aprender a utilizar el Ciclo while, el Ciclo inexacto.

Seba.

--

Idea y confección: Sebastián Duarte.

 **@sebateunmate**

Revisión y correcciones: Maximiliano Sar Fernández.

 **@maxiprograma**

--

Esta guía de ejercicios pertenece al "**Curso de Programación Desde Cero Nivel Universidad**" de **Maxi Programa**. Cualquier duda o sugerencia, no dejes de consultarla en los foros del aula virtual del curso.

<https://campusvirtualmaxiprograma.com>

<https://maxiprograma.com>